RELATÓRIO DO PROJETO POOFS

Programação Orientada aos Objetos 2024/2025

Desenvolvido por:

Diogo Rodrigues
2022257625

João Cruz
2022258375

Introdução:

O objetivo deste projeto foi desenvolver uma aplicação destinada à gestão financeira de uma empresa. A abordagem adotada para a sua concretização baseou-se na implementação de um menu interativo que permite ao utilizador escolher as operações que deseja realizar. Para garantir uma gestão eficiente, foram implementadas três listas: uma para clientes, outra para produtos e uma terceira para faturas e a aplicação foi projetada para funcionar manipulando estas listas, de modo a que seja possibilitada a realização das diversas funcionalidades do sistema.

A lista de clientes contém todos os clientes que já foram registados no sistema. Para criar um novo cliente ou alterar os seus dados, é necessário fornecer o seu número de contribuinte. Caso não exista nenhum cliente na lista com este número, então não será possível modificar os seus dados. Do mesmo modo, se já existir um cliente com o mesmo número de contribuinte não será possível adicionar um novo, dado que este é um identificador único de cada cidadão.

A lista de faturas é responsável por guardar todas as faturas, com os respetivos produtos, associadas aos vários clientes. Sempre que se cria uma nova fatura, esta deve ser associada a um cliente existente na lista de clientes. Caso contrário, não será possível concluir o seu registo. Para a alteração de qualquer fatura, é necessário informar o sistema de qual é o seu número. Se não existir nenhuma fatura com esse número, então não será possível modificar nenhum dado. Cada fatura tem a sua própria lista de produtos.

Por fim, a lista de produtos global que contém todos os itens disponíveis na loja. Para incluir um produto numa fatura, este deve constar previamente na lista de produtos. Caso contrário, a operação será recusada.

Explicitação do código:

Formatação

A classe ***Formatacao***, implementada como um **enum**, é responsável por facilitar a formatação visual da interface da aplicação. Nesta é definido um conjunto de constantes que representam códigos ANSI usados para aplicar estilos e cores ao texto exibido no terminal. O seu objetivo principal é melhorar a apresentação gráfica da aplicação, tornando-a mais atrativa e intuitiva, além de facilitar a leitura e compreensão das informações exibidas ao utilizador.

dei departamento de engenharia informática
1 2 9 0 FACULDADE DE CIÊNCIAS E TECNOLOGIA UNIVERSIDADE D COIMBRA

CategoriaInput

Esta classe do tipo enum define quais os diferentes tipos que o input pode ter. É usado na gestão do método *receberInput* de *POOFS.*

POOFS

A classe ***POOFS*** desempenha o papel central do projeto. Nesta classe, no construtor, são instanciadas as classes *Clientes*, *Faturas*, *Produtos* e *Leituras*, possibilitando o acesso e uso dos métodos e listas nelas definidos.

No método *main*, o ponto de entrada do programa, é criada uma instância da classe *POOFS*, que é então inicializada. Em seguida, o método *iniciar* é chamado, desencadeando o fluxo da aplicação. Neste método, o sistema executa a leitura de dados a partir de um ficheiro, utilizando o método *lerFicheiro* da classe *Leituras*. De seguida, é exibido um menu interativo ao utilizador, permitindo que ele escolha entre 11 operações disponíveis (considerando a opção 0 para encerrar o programa).

Cada entrada fornecida pelo utilizador é validada para garantir que corresponde a uma das opções disponíveis no menu. Em caso de erro, uma mensagem é exibida, informando o utilizador sobre a entrada inválida e solicitando uma nova resposta.

Após a execução de uma operação escolhida, o sistema questiona se o utilizador deseja regressar ao menu. Isso possibilita a realização de múltiplas operações numa mesma execução ou, alternativamente, o encerramento seguro do programa.

Quando se deseja recolher o que o utilizador escreve, chamamos o método *receberInput*, que, com um ciclo, apenas aceita um input até que este esteja de acordo com as regras fornecidas para cada formato. Para ajudar na validação dos inputs da localização e de um determinado texto, foram criados os métodos *isLocalizacaoValida* e *isTextoValido* que apenas aceitam as entradas do utilizador quando corresponde ao pretendido.

dei departamento de engenharia informática
1 2 9 0 FACULDADE DE CIÊNCIAS E TECNOLOGIA UNIVERSIDADE D COIMBRA

Caso o utilizador opte por encerrar o programa, as alterações realizadas são guardadas num ficheiro de objetos, com o auxílio do método *escreverObjeto* da classe *Leituras*, e uma mensagem informativa é exibida confirmando o término da execução.

Leituras

Na classe ***Leituras*** são instanciadas as classes *Clientes*, *Faturas* e *Produtos* no construtor, para que possam aqui ser utilizados os métodos nelas criados.

Quando no menu são escolhidas as opções "9- Exportar faturas" e "10- Importar faturas", são chamados os métodos *exportarFaturas* e *importarFaturas*, respetivamente: *exportarFaturas* recebe a lista de faturas e escreve as todas num ficheiro de texto, uma por cada linha, com a ajuda do método *toStringFicheiro* da classe Fatura; Do mesmo modo, *importarFaturas* recolhe as faturas que estão escritas num ficheiro de texto, com o auxílio do método *processarFaturas* e guarda-as numa lista. Esta lista substitui a lista principal de faturas através do respetivo *setter*.

Sempre que a execução do programa chega ao fim, é chamado o método *escreverObjeto* que recebe as listas de faturas, produtos e clientes e, por esta ordem, escreve-as como objetos no ficheiro de objetos "output.obj".

O método *lerFicheiro* é responsável pela inicialização dos dados do sistema. Se o ficheiro "output.obj" existir, os objetos nele contidos são lidos e atribuídos às listas de faturas, produtos e clientes utilizando setters, seguindo a mesma ordem em que foram guardadas. Caso o ficheiro não exista, o sistema lê e processa linha a linha o conteúdo do ficheiro de texto "input.txt". Para isto, seguimos as seguintes regras: linhas que começam com "CL" indicam a descrição de um novo cliente; as que começam com "F" contêm os dados de uma nova fatura; e as que não começam com nenhuma dessas strings descrevem novos produtos.

Para processar cada tipo de linha, são utilizados os seguintes métodos auxiliares: *processarCliente*, que lê os elementos da linha e preenche os atributos do objeto *Cliente* correspondente, depois adiciona esse cliente à lista principal de clientes no sistema. *processarFatura*, que analisa os dados da linha para criar uma nova fatura, associando-a a um cliente existente e aos produtos correspondentes. Se os dados forem válidos, a fatura é adicionada à lista principal de faturas; caso contrário, a fatura é descartada; *processarProduto*, que verifica o prefixo da linha para determinar o tipo de produto ("PA_TR", "PA_TI", "PA_TN", "PF_NP", "PF_P") e com base no tipo identificado, recolhe as partes subsequentes da linha para definir os atributos específicos do produto e adicioná-lo à lista.

Data

Esta classe tem o seu construtor com os atributos *dia*, *mes* e *ano*, e os métodos *toString* e *isDataValida*. Este último, determina se a data existe e se está entre o ano 2000 (possível data de abertura da loja) e 2024 (ano atual).

dei departamento de engenharia informática
1 2 1 9 0 FACULDADE DE CIÊNCIAS E TECNOLOGIA UNIVERSIDADE D COIMBRA

Classes base de todo o programa:

As classes *Cliente*, *Produto* e *Fatura* são inicializadas com a interface *Serializable*, permitindo que objetos dessas classes possam ser facilmente gravados e lidos a partir de um ficheiro de objetos.

Cliente

A classe ***Cliente*** possui um construtor que recebe os atributos nome, contribuinte e localização. Além disso, implementa o método *toString* para representar a instância da classe como uma string e inclui os métodos *getters* e *setters* para acessar e modificar os valores dos atributos.

Produto

A classe ***Produto*** tem um construtor que recebe os atributos *codigo*, *nome*, *descricao*, *quantidade* e *preco*. Também implementa o método *toString* e os métodos *getters* e *setters*. Adicionalmente, esta classe possui o método abstrato *calcularIva*, que é implementado nas subclasses de *Produto* para calcular o IVA de acordo com o tipo de produto.

Fatura

A classe ***Fatura*** contém o construtor com os seguintes atributos: *num*, *Cliente cliente*, *Data data* e *ArrayList<Produto> produtos*. Também inclui os *getters* e *setters* e 3 métodos *toString*: *toString*: Usado para listar as faturas na opção "6 - Listar faturas"; *toStringFaturaFormatada*: Para exibir a fatura de maneira formatada quando selecionado "7 - Visualizar faturas"; e *toStringFicheiro*: Usado quando se deseja exportar faturas para um ficheiro, em "9- Exportar faturas".

Para além disso, foi criado o método *calcularTotalBruto*, que, ao receber a lista de produtos que fazem parte da fatura, calcula o total bruto somando os preços dos produtos multiplicados pelas suas quantidades.

Organização e Manipulação das Listas

Para organizar o código de maneira mais eficiente, criaram-se as classes *Clientes*, *Faturas* e *Produtos*. Estas classes têm como objetivo manipular as listas das instâncias de *Cliente*, *Fatura* e *Produto*, facilitando a manipulação dos dados e o uso dos métodos dessas classes na aplicação.

Clientes

A classe ***Clientes*** é responsável pela gestão dos objetos de *Cliente* dentro do sistema que permite adicionar, editar e listar clientes. Esta manipula uma lista de instâncias da classe *Cliente* e proporciona métodos para interagir com esta lista. O único atributo que tem é uma ArrayList<*Cliente*> que armazena os clientes registados no sistema. Também tem o *setter* e o *getter*.

dei departamento de engenharia informática
1 2 9 0 FACULDADE DE CIÊNCIAS E TECNOLOGIA UNIVERSIDADE D COIMBRA

O método *procurarClientePorContribuinte* recebe um número de contribuinte e percorre a lista de clientes até encontrar um cliente com esse número fornecido, quando o encontrar devolve esse cliente. Se chegar ao final da lista, significa que nenhum cliente com este número associado foi encontrado e retorna null.

O método *novoCliente* permite a criação de um novo cliente, solicitando ao utilizador o nome, número de contribuinte e localização. As validações são feitas com os métodos de *POOFS isLocalizacaoValida* e *isTextoValido*. Este é chamado quando é escolhido "1 - Novo cliente".

adicionarCliente é utilizado para receber os atributos de um cliente, criá-lo e adicioná-lo à *listaClientes*. Este método é chamado no final do método anterior e no método de recolha de dados de clientes do ficheiro de texto, *processarCliente*.

O método *editarCliente* permite que o utilizador altere as informações de um cliente, como pretendido em "2 – Editar cliente". Primeiro, deve-se fornecer o número de contribuinte do cliente a ser editado. Caso o cliente seja encontrado, o utilizador é questionado sobre que dado quer alterar: nome, número de contribuinte ou localização. As atualizações são feitas no cliente utilizando os *setters* da classe Cliente. Após as alterações, o utilizador pode escolher se deseja realizar mais edições ou regressar ao menu.

O método *listarClientes* exibe na interface todos os clientes registados na lista, chamado quando se escolhe a opção "3 – Listar clientes". Se a lista estiver vazia, uma mensagem de erro é mostrada. Caso contrário, imprime todos os clientes, utilizando o método *toString* de cada cliente para formatar a saída.

Faturas

A classe *Faturas* é responsável por adicionar, editar e listar as faturas dentro do sistema. Esta manipula uma lista de instâncias da classe *Fatura* e, tal como de *Clientes* para *Cliente*, fornece métodos para facilitar a interação com essa lista. O único atributo que tem é uma ArrayList<*Faturas*> utilizado para armazenar todas as faturas registradas. Além disso, a classe possui métodos *getter* e *setter* para aceder e modificar esse atributo.

O método *procurarFatura* recebe um número de uma fatura e percorre a lista até encontrar uma fatura com esse número fornecido, quando o encontra devolve essa fatura. Se chegar ao final da lista, significa que nenhuma fatura com este número foi encontrada e retorna null.

O método *novaFatura* permite ao utilizador criar uma nova fatura. O número da fatura pode ser fornecido pelo utilizador ou gerado aleatoriamente com o método *gerarNumFatura*, dependendo do que o utilizador preferir. No caso de ser um número fornecido pelo utilizador, é verificado se não há mais nenhuma fatura com este número. Para continuar a criação da fatura, é pedido o número de contribuinte do cliente a que a mesma está associada, e verifica-se se este existe ou não esse cliente, a data em que foi efetuada a compra, que é verificada, e os códigos dos produtos que o utilizador deseja adicionar, também estes são validados. Este método é chamado quando a opção "4 – Nova fatura" é escolhida.

adicionarFatura, ao receber os atributos duma fatura, cria-a e adiciona-a à lista de faturas. Este método é chamado no final do método anterior e nos métodos de recolha de dados dos ficheiros de texto.

O método *editarFatura* permite que sejam efetuadas alterações aos constituintes duma fatura e é chamado ao ser escolhido "5 – Editar fatura". Depois de fornecer o número da mesma, o utilizador pode escolher quais atributos quer alterar: o seu número, o número de contribuinte do cliente, a data e se deseja adicionar ou remover algum(ns) produtos. As modificações são feitas na fatura através dos *setters* de *Fatura*. Após serem efetuadas, o utilizador pode decidir se quer fazer uma nova alteração na mesma ou não.

listarFaturas é utilizada quando a opção "6 – Listar faturas" é selecionada. Este, utilizando o *toString* de Fatura imprime-as uma a uma.

visualizarFatura, que dá resposta à escolha de "7 – Visualizar fatura", com a ajuda do método *toStringFaturaFormatada*, apresenta uma fatura com todas as informações de todos os seus atributos e utilizando os métodos de *Produto* determina a taxa e calcula o valor do IVA de cada produto e, de seguida os valores totais sem IVA, com IVA e do IVA.

apresentarEstatística, método chamado ao ser selecionado "8 – Apresentar estatística", calcula o número total de faturas e produtos nelas contidos, o total bruto, o total líquido e o valor do IVA.

Produtos

Esta classe funciona como as anteriores, tem o *setter* e *getter*, o método *adicionarProduto*, que o adiciona à lista fornecida, também tem o método *procurarProdutoCódigo* que procura um produto pelo seu código numa lista e o método *listarProdutos* que com a ajuda do *toString* de *Produto* imprime os vários produtos. A grande diferença é que enquanto que nas outras duas classes trabalhávamos sempre com as listas que continham todos os clientes e todas as faturas, neste são adicionados e listados produtos da lista que é enviada como argumento. Esta pode ser a lista total dos produtos, como por exemplo quando se usa o método *procurarProdutocodigo* para adicionar produtos a uma nova fatura e necessitamos de saber se ele existe na loja, ou pode ser a lista duma fatura, como por exemplo ao invocar o mesmo método mas para alterar os seus produtos constituintes.

Subclasses de Produto:

ProdutoAlimentar:

A subclasse ***ProdutoAlimentar*** tem um construtor semelhante ao construtor de *Produto*, apenas lhe é adicionada a variável booleana *biológico* que determina se o produto é biológico ou não. Nas suas subclasses, o cálculo do IVA depende sempre de se o produto é ou não biológico e da localização do cliente.

Na subclasse *ProdutoAlimentarTI* (taxa intermédia) foi incorporado um atributo extra denominado *CategoriaAlimentar*, representado como uma classe *enum*. Se o produto pertence à categoria *VINHO*, será adicionado o respetivo acréscimo ao IVA.

dei departamento de engenharia informática
1 2 9 0 FACULDADE DE CIÊNCIAS E TECNOLOGIA UNIVERSIDADE D COIMBRA

Em *ProdutoAlimentarTN* (taxa normal), não há atributos adicionais, tornando então o cálculo do IVA mais direto e simples.

Já em *ProdutoAlimentarTR* (taxa reduzida), são adicionadas as *Certificações*, também representada como classe *enum*. Se um produto tiver as quatro certificações possíveis, então também lhe será aplicado um desconto no IVA.

ProdutoFarmacia:

A subclasse ***ProdutoFarmacia*** utiliza o mesmo construtor de Produto.

Na sua subclasse *ProdutoFarmaciaPrescrito* é incluído o atributo *medico*, que armazena o nome do médico responsável pela prescrição do produto. Neste caso, o cálculo do IVA depende exclusivamente da localização do cliente.

Já para *ProdutoFarmaciaNaoPrescrito* é acrescentada a *CategoriaFarmacia*, também representado como uma classe *enum*. No seu cálculo do IVA, a localização do cliente não modifica nada, e no caso de ser da categoria *ANIMAL*, então a taxa é reduzida.

```
● Ficheiro 'output.obj' encontrado.
● As listas foram escritas no ficheiro 'output.obj' com sucesso.
  Execução terminada

Process finished with exit code 0
```